PCCST503 – MACHINE LEARNING

USER MANUAL

Safe Semantic Planner in a Finite Cartesian State Space

Submitted by: Shazia Habeeb
Department of Computer Science and Engineering
B.Tech Computer Science and Engineering
Academic Year: 2026

1. Purpose

This user manual explains how to compile, run, and use the Safe Semantic Planner implemented in C++. The program finds paths between states while avoiding bad states and demonstrates dynamic changes such as transition failures, goal updates, and newly added transitions.

2. Requirements

• Windows computer
• Visual Studio Code (or another C++ development environment)
• MinGW-w64/MSYS2 GCC compiler with g++ available
• The project source file: main.cpp

3. Project Files

• main.cpp – complete C++ implementation.
• README.md – project description and basic instructions.
• Assignment1_Report.pdf – design and experimental report.
• planner.exe – local compiled executable; it does not need to be uploaded to GitHub.

4. Compilation

Open the project folder in Visual Studio Code and open the integrated terminal. Compile using:

```
g++ main.cpp -o planner.exe
```

If there are no compiler errors, planner.exe will be generated in the project folder.

5. Running the Program

Run the program in the VS Code PowerShell terminal using:

.\planner.exe

The main menu contains:

1. Basic Reachability
2. Bad State Avoidance
3. Safety Margin
4. Dynamic Transition
5. Goal Update
6. Transition Addition
7. Run All Test Cases
8. Exit

6. Menu Options

Option 1 – Basic Reachability: tests whether the planner can find the basic path from the initial state to the goal.

Option 2 – Bad State Avoidance: tests whether a path containing a bad state is rejected and an alternative safe path is selected.

Option 3 – Safety Margin: tests how the planner considers safety distance when choosing between valid paths.

Option 4 – Dynamic Transition: a transition is made unavailable and the planner generates an alternative path.

Option 5 – Goal Update: changes the goal and demonstrates replanning.

Option 6 – Transition Addition: adds a shortcut and checks whether the planner discovers the improved route.

Option 7 – Run All Test Cases: runs all six test cases sequentially.

Option 8 – Exit: terminates the program.

7. Understanding the Output

The program displays State Path, Transition Path, Total Cost, Minimum Safety Distance, Cumulative Reliability, Explored States, Planning Time, and Bad States Visited. The expected number of bad states visited is zero.

8. Demonstrated Test Results

Test Case 1: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$, total cost 3.000, bad states visited 0.

Test Case 2: $0 \rightarrow 3 \rightarrow 4 \rightarrow 5$, total cost 3.000, bad states visited 0.

Test Case 3: $0 \rightarrow 3 \rightarrow 4 \rightarrow 5$, total cost 9.000, minimum safety distance 1.118, bad states visited 0.

Test Case 4: Before failure, 0 → 1 → 4 with cost 2.000. After transition failure, 0 → 2 → 3 → 4 with cost 6.000.

Test Case 5: Original goal gives 0 → 1 → 2 with cost 2.000. After the goal changes to state 4, the planner produces 0 → 3 → 4 with cost 4.000.

Test Case 6: Before shortcut, 0 → 1 → 2 → 3 with cost 6.000. After adding the shortcut, the planner uses 0 → 3 with cost 1.000.

9. Troubleshooting

If 'g++ is not recognized' appears, ensure MSYS2/MinGW-w64 GCC is installed and C:\msys64\ucrt64\bin is in the Windows PATH. Restart VS Code after changing PATH.

If the program does not compile, make sure the file is named main.cpp and not main.cpp.txt.

If planner.exe is not found, compile the program first.

10. Demonstration Procedure

1. Open main.cpp in VS Code.
2. Open the terminal.
3. Compile using g++ main.cpp -o planner.exe.
4. Run using .\planner.exe.
5. Select option 7.
6. Explain the six test cases and displayed metrics.
7. Point out that Bad States Visited remains zero.
8. Explain that Test Cases 4–6 demonstrate dynamic changes and replanning.

11. States and Diagrams

The program represents states and transitions internally and prints state IDs and transition IDs as paths. It does not draw graphical diagrams on screen. The test graphs can, however, be represented visually in the report. For example, Basic Reachability is S → A → B → G. Bad State Avoidance can be represented as S → A → X → G (X is bad) and S → C → D → G (selected safe route).